计算机组成期中复习。

第一章——衡量CPU的效率
第二章——RISCV指令
第三章——算数运算
- 加减法
- 乘法
- 除法
- 浮点加法

第四章——CPU设计
- 单周期CPU
- 流水线CPU
  - 冒险
    - 数据冒险☆
    - 控制冒险
    - 结构冒险
  - 总结
- Trap
- Multiple Issue

第一章——衡量CPU的效率

CPU的8个设计原则：

1. Design for Moore's Law
2. Use Abstraction to Simplify Desgin
3. Make the Common Case Fast
4. Performance via Parallelism
5. Performance via Pipelining
6. Performance via Prediction
7. Hierarchy of Memories
8. Dependability via Redundancy

衡量CPU的效率：

CPU时间 = 指令数量×IPC = IC×CPI×时钟周期长度 = IC×CPI/时钟频率

CPI，指令平均时钟周期数clock cycle per instruction

IC，指令数量，Instruction count

第二章——RISCV指令

基础的RV64I一共有6种类型的指令：

31 30 25	24 21	20 19	15 14 12	11 8	7 6 0	
funct7	rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]		rs1	funct3	rd	opcode	I-type
imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12] imm[10:5]	rs2	rs1	funct3	imm[4:1] imm[11]	opcode	B-type
imm[31:12]				rd	opcode	U-type
imm[20] imm[10:1] imm[11]		imm[19:12]		rd	opcode	J-type

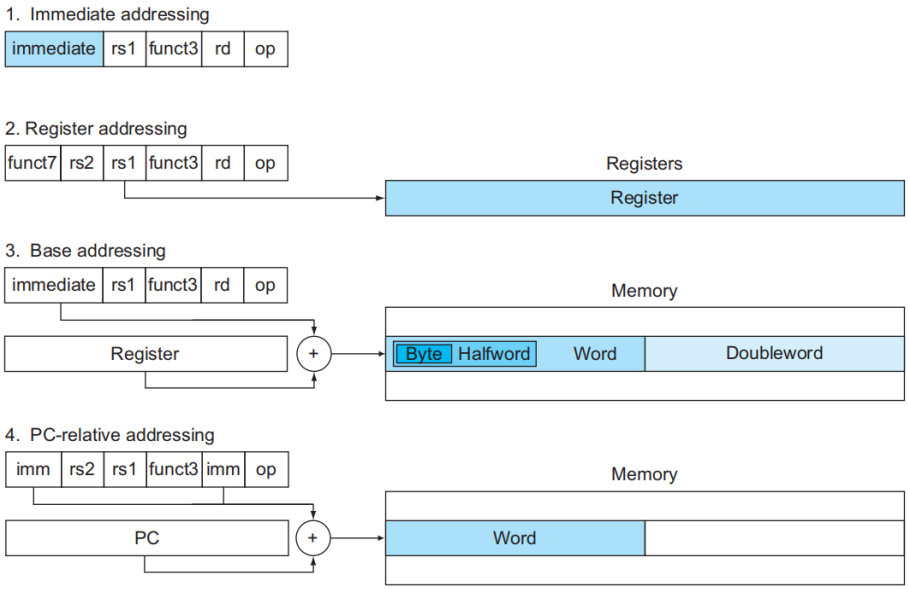
31 25 24	20 19	15 14	12 11	7 6 0	
imm[31:12]			rd	0110111	U lui
imm[31:12]			rd	0010111	U auipc
imm[20|10:1|11|19:12]			rd	1101111	J jal
imm[11:0]	rs1	000	rd	1100111	I jalr
imm[12|10:5]	rs2	rs1	000	imm[4:1|11]	1100011	B beq
imm[12|10:5]	rs2	rs1	001	imm[4:1|11]	1100011	B bne
imm[12|10:5]	rs2	rs1	100	imm[4:1|11]	1100011	B blt
imm[12|10:5]	rs2	rs1	101	imm[4:1|11]	1100011	B bge
imm[12|10:5]	rs2	rs1	110	imm[4:1|11]	1100011	B bltu
imm[12|10:5]	rs2	rs1	111	imm[4:1|11]	1100011	B bgeu
imm[11:0]		rs1	000	rd	0000011	I lb
imm[11:0]		rs1	001	rd	0000011	I lh
imm[11:0]		rs1	010	rd	0000011	I lw
imm[11:0]		rs1	100	rd	0000011	I lbu
imm[11:0]		rs1	101	rd	0000011	I lhu
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	S sb
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	S sh
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	S sw
imm[11:0]		rs1	000	rd	0010011	I addi
imm[11:0]		rs1	010	rd	0010011	I slti
imm[11:0]		rs1	011	rd	0010011	I sltiu
imm[11:0]		rs1	100	rd	0010011	I xori
imm[11:0]		rs1	110	rd	0010011	I ori
imm[11:0]		rs1	111	rd	0010011	I andi
0000000	shamt	rs1	001	rd	0010011	I slli
0000000	shamt	rs1	101	rd	0010011	I srli
0100000	shamt	rs1	101	rd	0010011	I srai
0000000	rs2	rs1	000	rd	0110011	R add
0100000	rs2	rs1	000	rd	0110011	R sub
0000000	rs2	rs1	001	rd	0110011	R sll
0000000	rs2	rs1	010	rd	0110011	R slt
0000000	rs2	rs1	011	rd	0110011	Rsltu
0000000	rs2	rs1	100	rd	0110011	R xor
0000000	rs2	rs1	101	rd	0110011	R srl
0100000	rs2	rs1	101	rd	0110011	R sra
0000000	rs2	rs1	110	rd	0110011	R or
0000000	rs2	rs1	111	rd	0110011	R and
0000 pred	succ	00000	000	00000	0001111	I fence
0000 0000	0000	00000	001	00000	0001111	I fence.i
000000000000		00000	000	00000	1110011	I ecall
000000000001		00000	000	00000	1110011	I ebreak
csr		rs1	001	rd	1110011	I csrrw
csr		rs1	010	rd	1110011	I csrrs
csr		rs1	011	rd	1110011	I csrrc
csr		zimm	101	rd	1110011	I csrrwi
csr		zimm	110	rd	1110011	I cssrrsi
csr		zimm	111	rd	1110011	I csrrci

图 2.3：RV321 带有指令布局，操作码，格式类型和名称的操作码映射。（此图基于[Waterman and Asanovi'c 2017]的表 19.2。）

学会汇编看这些东西都没什么问题。不过需要注意的是，RISCV的寻址方式。

1. PC相对寻址。无论是beq，还是jal，都是以PC当前的地址作为基址来进行相对偏移寻址。这个和X86是不同的。至于原理，看下面CPU的设计图就一目了然了。另外，RISCV的指令要求是2字节对齐的，所以你看到那些跳转指令的立即数都是从1开始的。
2. 基址寻址，操作数在内存中，地址是寄存器和指令中的常量之和。`jalr rd, offset(rs1)`，是把pc=rs1+offset。`ld rs,offset(rd)`
3. 寄存器寻址，操作数全部都在寄存器中。比如`add rd,rs1,rs2`，

4. 立即数寻址，操作数是指令本身的常量。比如`addi rd,rs,imm`。`sd rs,offset(rd)`应该是吧?



还有两条加锁指令:

`lr.d rd, (rs1) x[rd] = LoadReserved64(M[x[rs1]])`加载保留双字(Load-Reserved Doubleword). R-type, RV64A.
从内存中地址为 x[rs1]中加载八个字节，写入 x[rd]，并对这个内存双字注册保留。

`sc.d rd, rs2, (rs1) x[rd] = StoreConditonal64(M[x[rs1], x[rs2])`条件存入双字(Store-Conditional Doubleword). R-type, RV64A only
如果内存地址 x[rs1]上存在加载保留，将 x[rs2]寄存器中的 8 字节数存入该地址。如果存入成功，向寄存器 x[rd]中存入 0，否则存入一个非 0 的错误码。

据说从C翻译汇编/汇编翻译成C是必考题啊。

第三章——算数运算

这一章节我在书上做了很多笔记，就不搬运了。

加减法

这个没啥好说的，数字逻辑就学过了。

溢出的判断条件 `次高位进位 xor 最高位进位 = 1`，说明有符号数溢出。

乘法

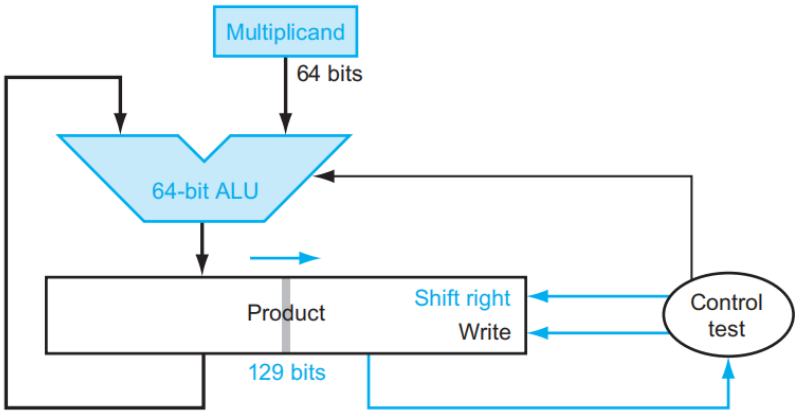


FIGURE 3.5 Refined version of the multiplication hardware. Compare with the first version in Figure 3.3. The Multiplicand register and ALU have been reduced to 64 bits. Now the product is shifted right. The separate Multiplier register also disappeared. The multiplier is placed instead in the right half of the Product register, which has grown by one bit to 129 bits to hold the carry-out of the adder. These changes are highlighted in color.

除法

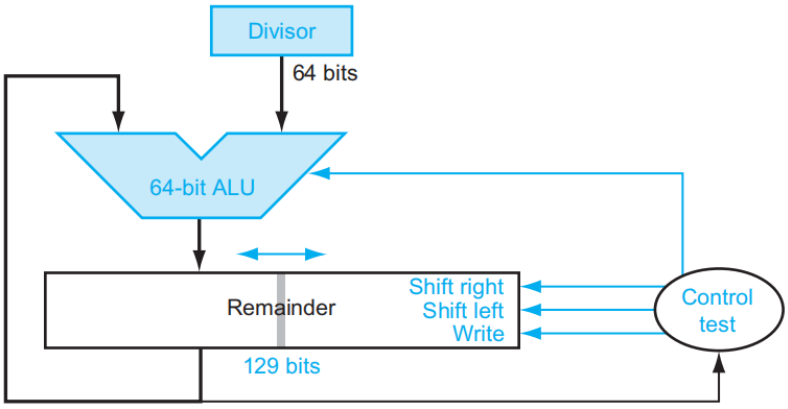


FIGURE 3.11 An improved version of the division hardware. The Divisor register, ALU, and Quotient register are all 64 bits wide. Compared to Figure 3.8, the ALU and Divisor registers are halved and the remainder is shifted left. This version also combines the Quotient register with the right half of the Remainder register. As in Figure 3.5, the Remainder register has grown to 129 bits to make sure the carry out of the adder is not lost.

浮点加法

首先要知道IEEE的浮点数表示规范。

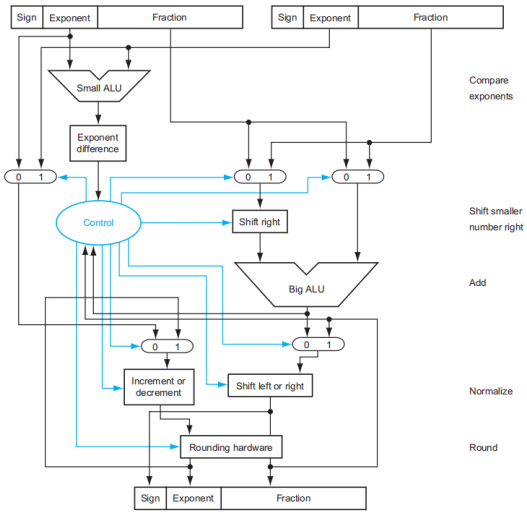


FIGURE 3.15 Block diagram of an arithmetic unit dedicated to floating-point addition. The steps of Figure 3.14 correspond to each block, from top to bottom. First, the exponent of one operand is subtracted from the other using the small ALU to determine which is larger and by how much. This difference controls the three multiplexors; from left to right, they select the larger exponent, the significand of the smaller number, and the significand of the larger number. The smaller significand is shifted right, and then the significands are added together using the big ALU. The normalization step then shifts the sum left or right and increments or decrements the exponent. Rounding then creates the final result, which may require normalizing again to produce the actual final result.

最后要知道提高精度的一些办法。

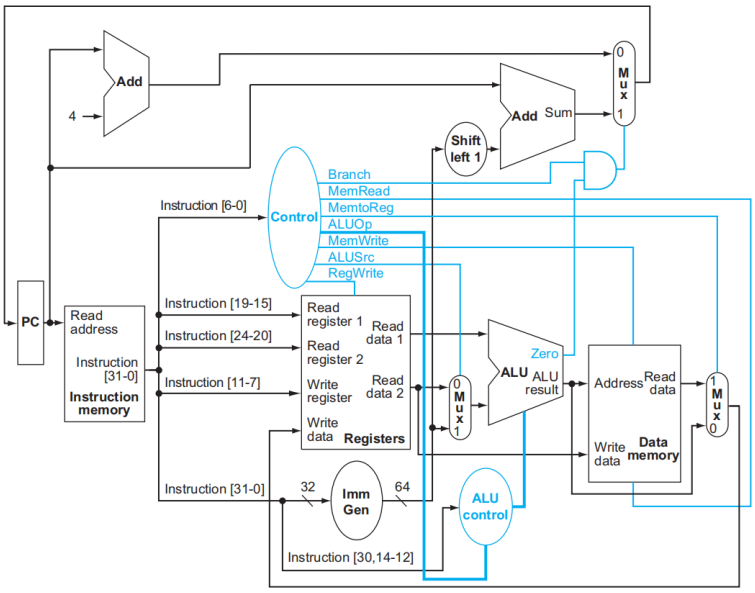
第四章——CPU设计

书上的设计满足的指令有：

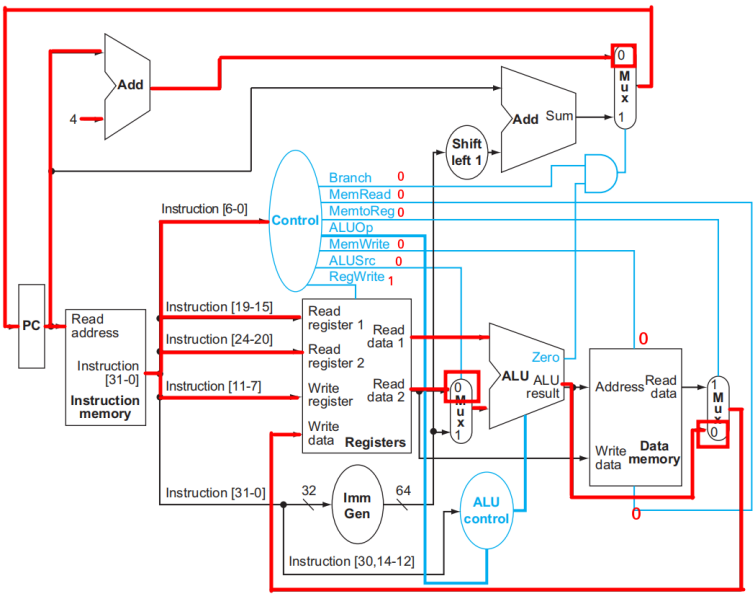
- R-type：add sub ...
- I-type：addi, subi, ... ld
- S-type：sd
- B-type：beq

我们的实验加上了Jal

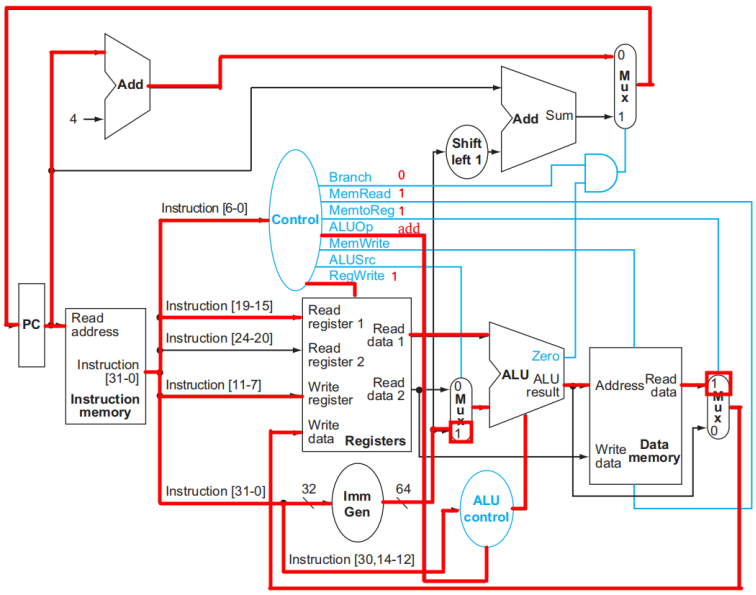
单周期CPU



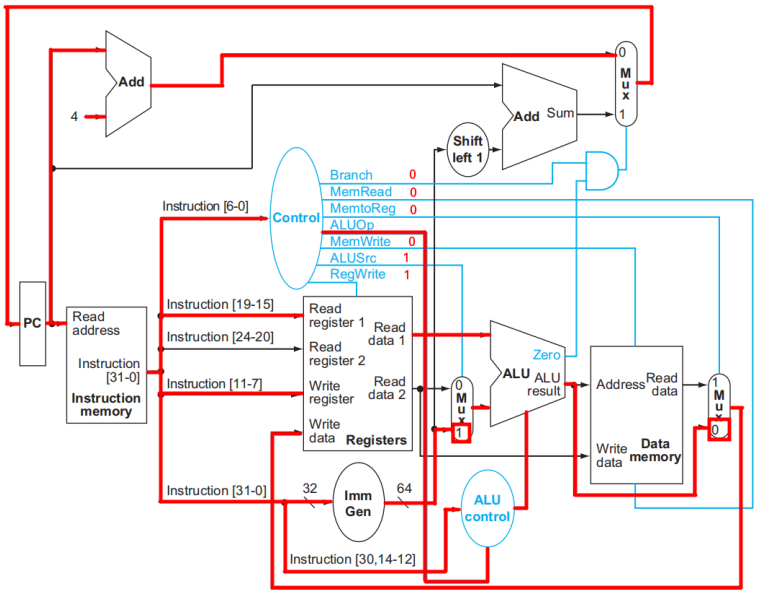
R-Type指令的数据通路。包括add,sub,and,or,xor,not,slt,shr



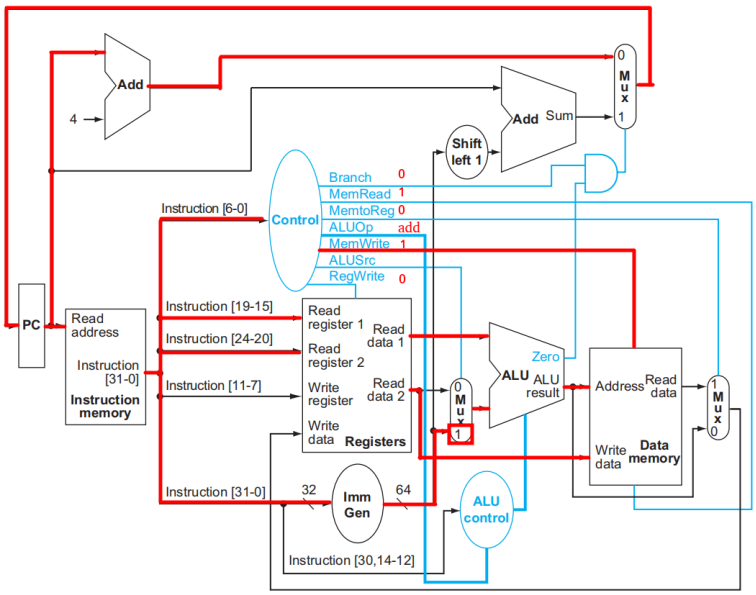
LD：



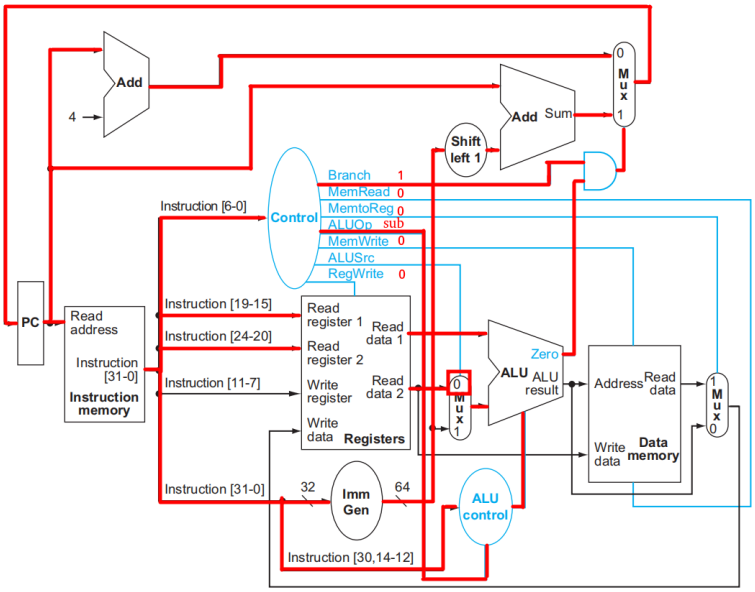
I-Type：addi, subi, ori,



S-type：sd



B-type：beq

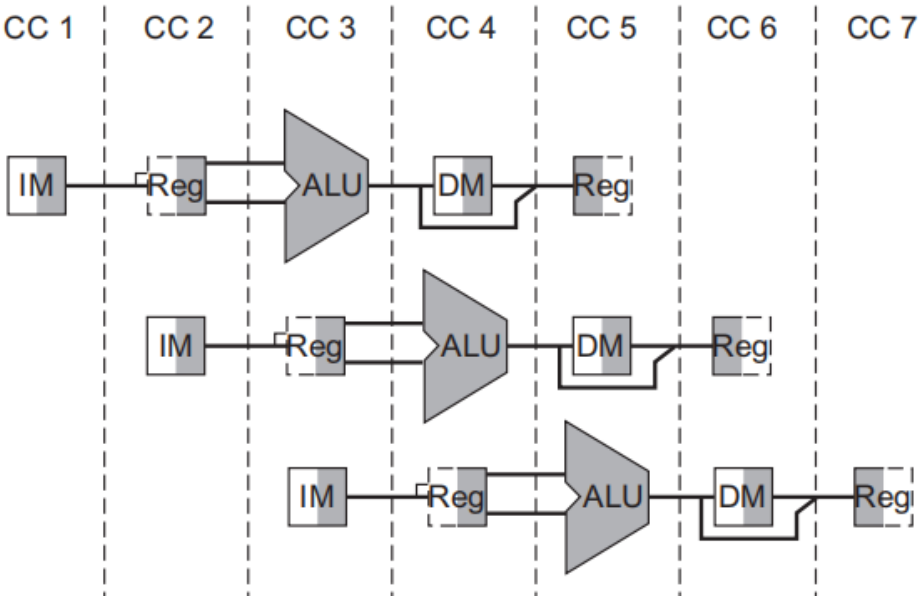

Add
4
Add
Sum
Shift left 1
M u x
0
1
Branch 1
MemRead 0
MemtoReg 0
ALUOp sub
MemWrite 0
ALUSrc
RegWrite 0
Control
Instruction [6-0]
PC
Read address
Instruction [31-0]
Instruction memory
Instruction [19-15]
Instruction [24-20]
Instruction [11-7]
Read register 1
Read register 2
Write register
Write data
Read data 1
Read data 2
Registers
Zero
ALU
ALU result
Address
Read data
Write data
Data memory
Instruction [31-0]
32
Imm Gen
64
ALU control
Instruction [30,14-12]

流水线CPU

经典的五阶段流水线CPU：

1. IF: Instruction fetch
2. ID: Instruction decode and register file read
3. EX: Execution or address calculation
4. MEM: Data memory access
5. WB: Write back

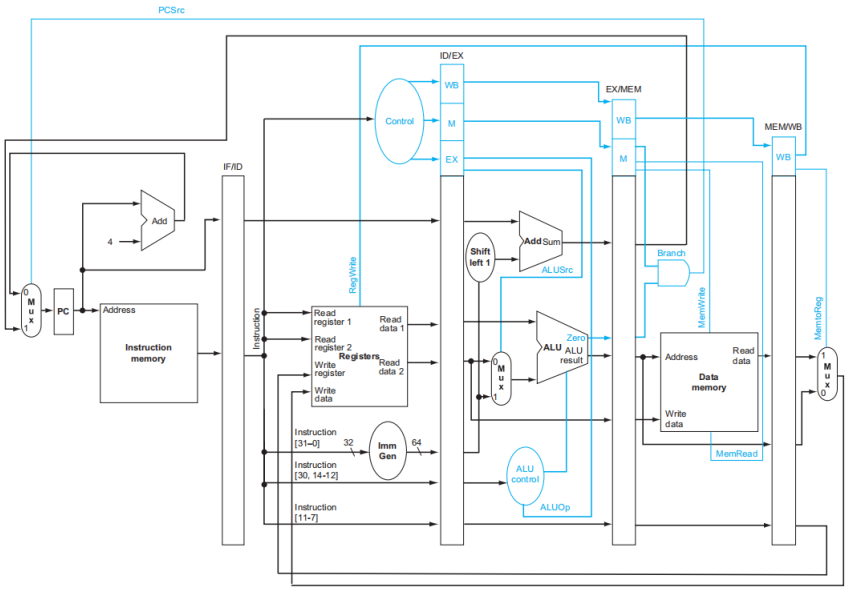
IF - ID - EX - MEM - WB



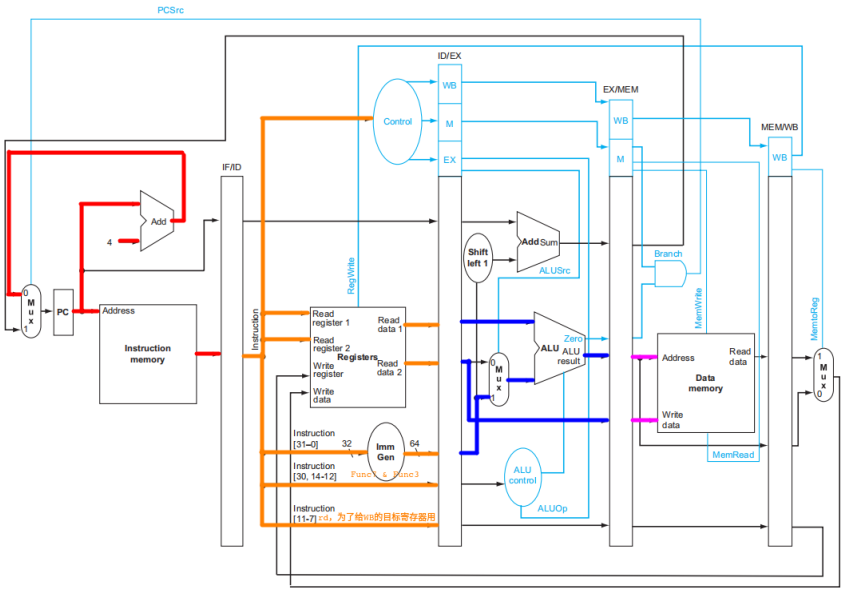
这是没有考虑冒险的Pipeline CPU设计图。

我们要保存很多信息以备之后使用。比如要保存rd作为写回阶段的目标寄存器，比如ld。

Control信号中，在ID阶段除了ALU Control都可以确定了。所以ID阶段把control信号传递给后面的阶段，这样就不用把整条指令往后面传递了。



可以来看一下Sd的数据通路，



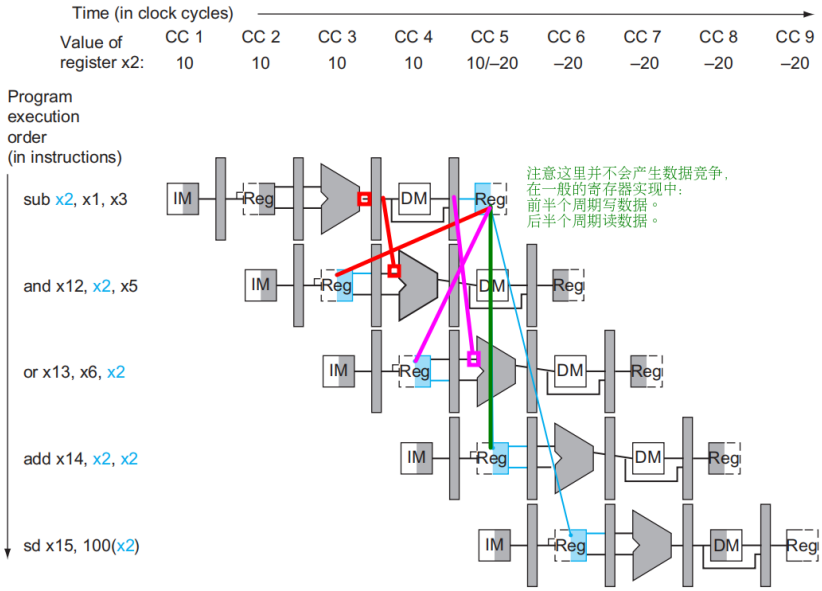
- 我们要传递到WB的信号（MEM/WB保存的信号）：RegWrite决定要不要写回，MemtoReg决定是从Mem还是ALU中写回。
- 我们要传递到MEM的信号（EX/MEM保存的信号）：要保存上述的两个信号。Branch、MemRead、MemWrite需要。

冒险

数据冒险☆

红色：第一种数据竞争。但其实在第一条指令的EX后数据就已经产生，在第二条指令的EX数据才真正被使用。所以可以将数据forward。也就是说，在ALU的Src1和Src2之前要加上MUX来选择。

粉色：第二种数据竞争。同样的在第一条指令的EX后数据就已经产生，在第三条指令的EX数据才真正被使用。也可以将数据forward



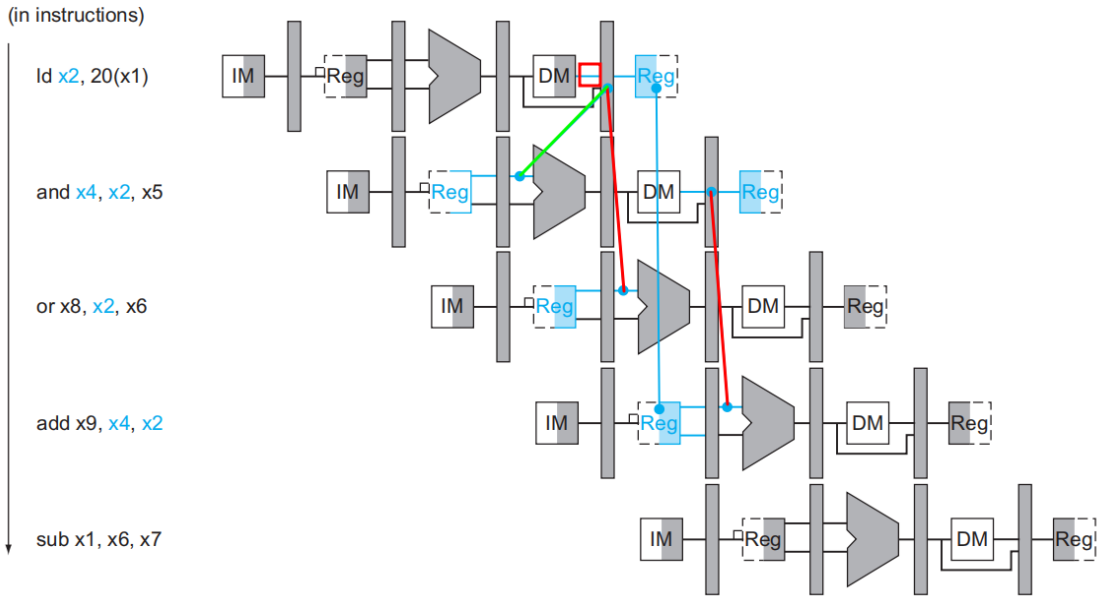
红色冲突**EX冒险**的解决方案：

```
if (EX/MEM.RegWrite                                  //一定是之后要写回才要forward
and EX/MEM.RegWrite != 0                             //x0不需要forward，否则反而是错的
and EX/MEM.RegisterRd == ID/EX.RegisterR1)           //发生了冲突
ForwardA = 10;

if (EX/MEM.RegWrite
and EX/MEM.RegWrite != 0
and EX/MEM.RegisterRd == ID/EX.RegisterR2)
ForwardB = 10;
```

粉色冲突的**MEM冒险**解决方案：

```
if (EX/MEM.RegWrite                                  //一定是之后要写回才要forward
and EX/MEM.RegWrite != 0                             //x0不需要forward，否则反而是错的
and not (EX/MEM.RegWrite and EX/MEM.RegWrite != 0 and EX/MEM.RegisterRd == ID/EX.RegisterR1)    //如果是红色冲突发生，红色优先。
                                                     //在上图中，就是第二条指令和第三条指令之间出现了冒险
and EX/MEM.RegisterRd == ID/EX.RegisterR1)           //发生了冲突
ForwardA = 01;

if (EX/MEM.RegWrite
and EX/MEM.RegWrite != 0
and not (EX/MEM.RegWrite and EX/MEM.RegWrite != 0 and EX/MEM.RegisterRd == ID/EX.RegisterR2)
and EX/MEM.RegisterRd == ID/EX.RegisterR2)
ForwardB = 01;
```

数据冒险的另一种情况：ld指令。



红色冒险可以用forward的办法和之前一样解决，但是我们显然无法把绿色冒险forward。因为ld指令的数据在MEM/WB阶段才真正产生。所以对于绿色冒险，我们必须用stall的办法：

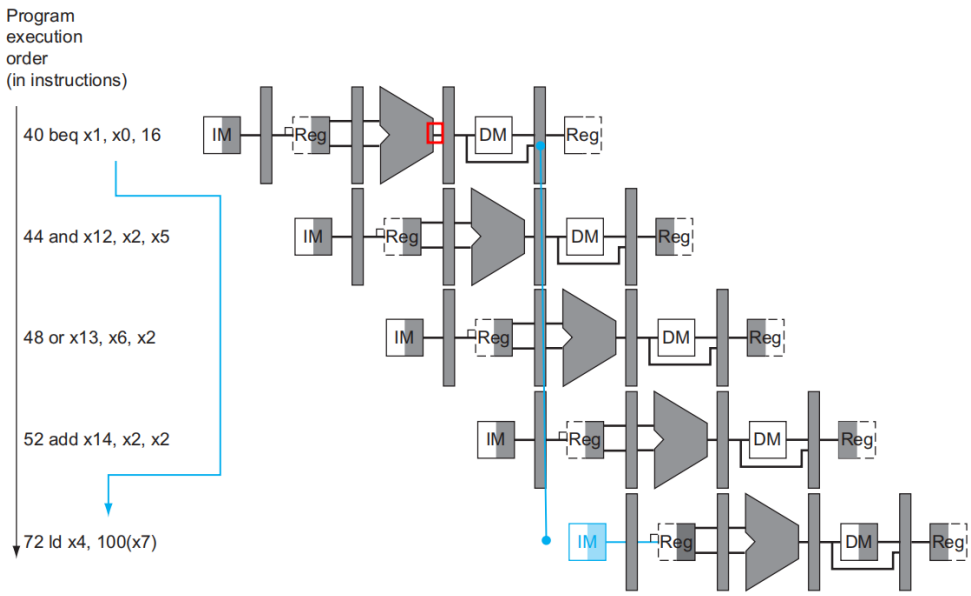
```
if (ID/EX.MemRead and                          //我发现我要Load内容了，趁早发现，否则就2条指令开始执行了
    ID/EX.RegisterRd = IF/ID.RegisterRs)       //要load的目标和要用的寄存器冲突了，Rs1/Rs2
    stall the pipeline
```

如果我们多stall几个就可以完全避免问题，但事实上我们就stall一个周期，之后再用forward来解决问题。

```
if (MEM/WB.RegisterRd == ID/EX.RegisterRs
    MEM/WB.RegisterRd != 0)
    forward
```

控制冒险

如图所示，我们到EX/MEM阶段才知道我们是不是需要跳转，这个时候好几条指令都执行完了。



有这些解决方案：

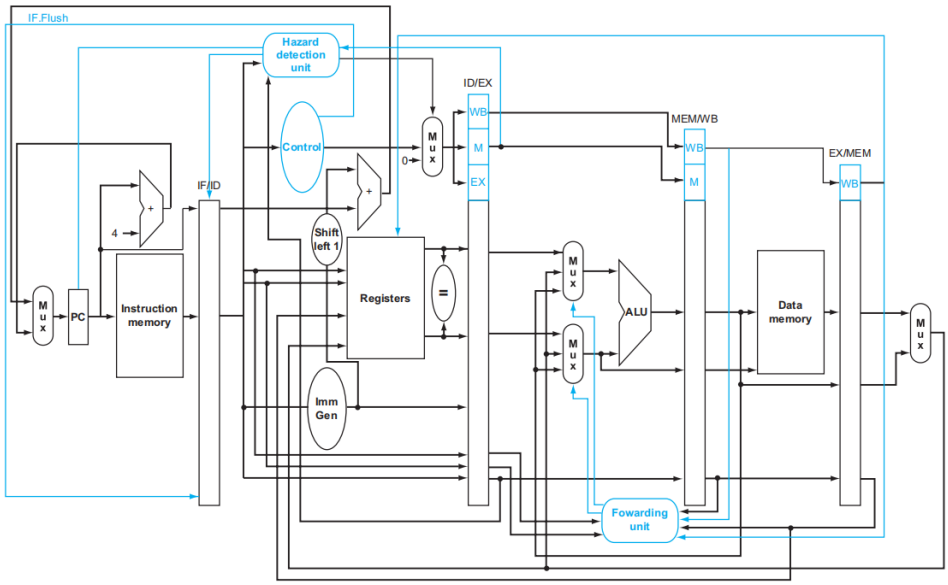
1. 假设分支不发生。这样分支不发生的时候没有代价；如果分支发生，我们需要丢弃三条指令的结果。因为他们还没有到达MEM和WB，所以直接把他们的信号清空即可。还可以更早的时候EX/MEM知道，这样只需要丢弃两条指令。
2. 事实上我们还可以更早的时候知道，在ID/EX的时候就可以知道，这样就只需要丢弃一条指令。但是这个就十分复杂了。
    1. 这意味着之前的forward要考虑更多，因为可能都要forward到ID阶段用了。
    2. 在ID阶段添加检测是否需要跳转的XOR
3. 动态分支预测。用更高级的策略预测是否分支。
4. 停顿到出结果为止23333。

结构冒险

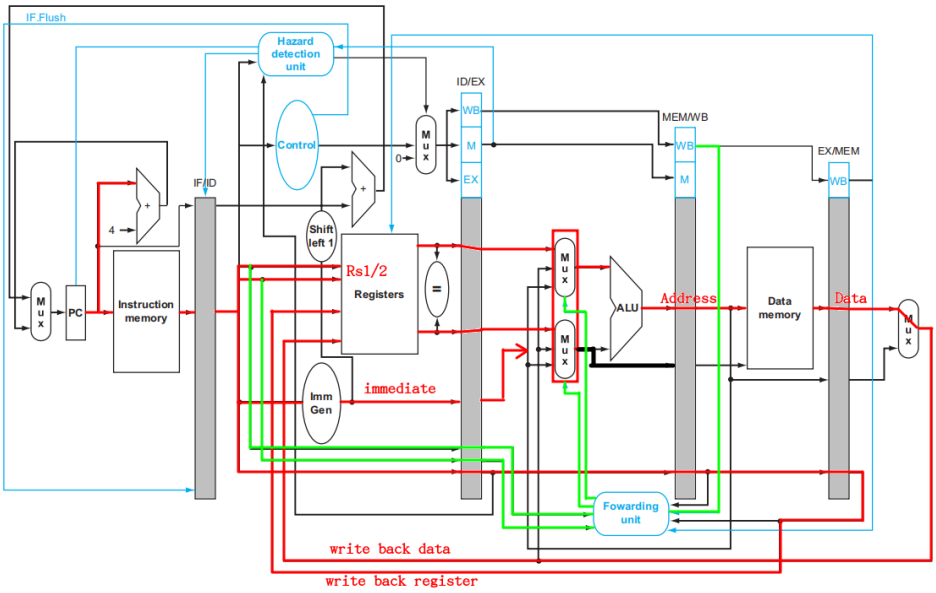
硬件不支持多条指令在同一个时钟周期执行。我们在设计流水线CPU时就应该避免出现结构冒险。

总结

这是最终的流水线CPU设计图



举一个ld的例子，来说明如何工作。我觉得上面那个图不太对，虽然下面这个图也是一个半成品。


IF.Flush
Hazard detection unit
ID/EX
WB
M
EX
MEM/WB
WB
M
EX/MEM
WB
Control
Mux
0
IF/ID
4
+
+
Shift left 1
Rs1/2
Registers
=
Mux
PC
Instruction memory
Imm Gen
immediate
Mux
Mux
ALU
Address
Data memory
Data
Mux
Fowarding unit
write back data
write back register

Trap

Trap包括Exception和Interrupt。是更高级的控制冒险了。

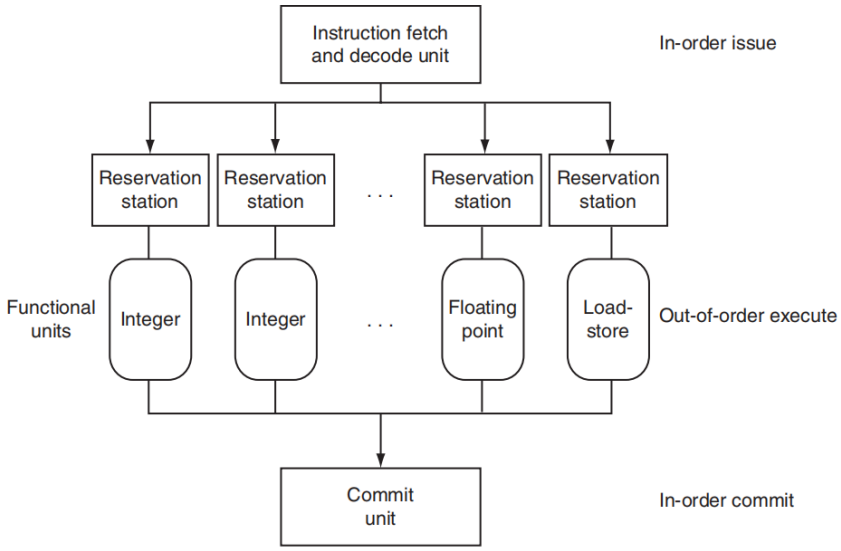
一个是确定地址+scause的异常处理，一个是根据不同类型跳转到不同地址的异常处理。

Multiple Issue

如果出现了冒险，就不好双发了。

静态多发：由编译器来打包**发射指令包**（一个周期同时执行的指令们），书上举了例子是lw/sw和算数指令的一起发射，编译器要负责不能让他们冲突。

动态多发处理器，也叫做**超标量superscalar**处理器。

经典的动态多发示意图：Reservation station存放指令的操作和所需的操作数，只要他们准备好，并且Funcitional units准备就绪，就可以执行指令。指令执行结束，结果被传送给Reservation station中正在等待使用该结果的指令，同时也传送到Commit unit中进行保存。Commit Unit中保存了已完成指令的执行结果，并在指令真正提交的时候才使用他们更新寄存器或写入内存。

位于Commit unit的buffer，通常叫做reorder buffer，也可以为其他指令提供操作数。一旦指令提交，寄存器就得到更新，就可以像正常流水线一样直接从寄存器中获取最新的数据。